Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Common Log Formats

Stellar Cyber supports numerous log formats and combinations of formats that log sources on the network send to modular sensors. This page presents several of the most common log formats that Stellar Cyber parsers support. For each log format, there's a brief introduction, an example log, and a breakdown of the components of the log. Following this is a summary of the differences between the various formats. Finally, there's a section that covers how different formats and different parsing and processing techniques like regex and Logstash can be combined and an example with a breakdown of its components.

- **CEF (Common Event Format)**
- **Delimited Text Formats (CSV, TSV, and Pipe-Separated Values)**
- **JSON (JavaScript Object Notation)**
- **Key-Value Pairs**
- **LEEF (Log Event Extended Format)**
- **Syslog RFC 3164**
- **Syslog RFC 5424**
- **Summary of Differences**
- **Log Format Combinations**

The following are the major formats of data logs sent to Stellar Cyber. For each format, there's an example of the same event so you can more easily spot format differences: a successful user login from 192.168.1.10 to 192.168.1.20 at 12:00:01 PM UTC on August 4, 2024.

CEF (Common Event Format)

CEF is a log format designed for interoperability between different security products and Security Information and Event Management (SIEM). It is structured and easy to parse, with fields like severity, event name, and source/destination IP addresses.

Example of a CEF-formatted log message

```
CEF:0|SecurityVendor|Product|1.0|100|LoginSuccess|5|src=192.168.1.10
dst=192.168.1.20 suser=admin msg=User login successful
```

Breakdown of the message

Version Information: `CEF:0` (indicates the version of the CEF format).

Device Information:

- Vendor: `SecurityVendor`
- Product: `Product`
- Version: `1.0`

Event Information:

- Signature ID: `100`
- Name: `LoginSuccess` (event name)
- Severity: `5` (severity level from 0 to 10)

Extension (Key-Value Pairs):

- Source IP: `src=192.168.1.10`
- Destination IP: `dst=192.168.1.20`
- User: `suser=admin`
- Message: `msg=User login successful`

Delimited Text Formats (CSV, TSV, and Pipe-Separated Values)

Delimited text formats are simple, text-based methods in which data fields are separated by specific characters. These formats are commonly used for exporting and importing data between applications:

- Comma-separated values (CSV): Fields are separated by commas.
- Tab-separated values (TSV): Fields are separated by tabs.
- Pipe-separated values: Fields are separated by the pipe (|) character.

While CSV is the most common, TSV and pipe-separated values are useful alternatives when the data sometimes contain commas or tabs. Other methods of separating values include space-separated values (SSV) and grave accent-separated (`) values.

Comma-Separated Values

Example of a CSV-formatted log message

```
timestamp,source_ip,destination_ip,username,event,description 2024-08-04T12:00:01Z,192.168.1.10,192.168.1.20,admin,LoginSuccess,User login successful
```

Breakdown of the message

Header Row: `timestamp,source_ip,destination_ip,username,event,description` (defines the fields)

Data Row: `2024-08-04T12:00:01Z,192.168.1.10,192.168.1.20,admin,LoginSuccess,User login`

`successful` (values correspond to the headers)

Tab-Separated Values

Example of a TSV-formatted log message

```
timestamp source_ip destination_ip username event description 2024-08-04T12:00:01Z 192.168.1.10 192.168.1.20 admin LoginSuccess User login successful
```

Breakdown of the message

Header Row: `timestamp source_ip destination_ip username event description` (defines the fields)

Data Row: `2024-08-04T12:00:01Z 192.168.1.10 192.168.1.20 admin LoginSuccess User login successful` (values correspond to the headers)

Pipe-Separated (|) Values

Example of a pipe-separated values log message

```
timestamp|source_ip|destination_ip|username|event|description 2024-08-04T12:00:01Z|192.168.1.10|192.168.1.20|admin|LoginSuccess|User login successful
```

Breakdown of the message

Header Row: `timestamp|source_ip|destination_ip|username|event|description` (defines the fields)

Data Row: `2024-08-04T12:00:01Z|192.168.1.10|192.168.1.20|admin|LoginSuccess|User login successful` (values correspond to the headers)

JSON (JavaScript Object Notation)

JSON is a lightweight data format that uses key-value pairs within a structured, human-readable text format. It's widely used in web applications and log management systems.

Example of a JSON-formatted log message

```
{
"timestamp": "2024-08-04T12:00:01Z",
"source_ip": "192.168.1.10",
"destination_ip": "192.168.1.20",
"username": "admin",
"event": "LoginSuccess",
"description": "User login successful"
}
```

Breakdown of the message

Key-Value Pairs: Structured as an object with fields and values.

- Timestamp: `"timestamp": "2024-08-04T12:00:01Z"`
- Source IP: `"source": "192.168.1.10"`
- Destination IP: `"destination_ip": "192.168.1.20"`
- Username: `"username": "admin"`
- Event: `"event": "LoginSuccess"`
- Description: `"description": "User login successful"`

Key-Value Pairs

This format represents data as a series of key-value pairs, which are easy to parse and search. It is commonly used in configuration files and log data.

Example of a log message with key-value pairs

```
timestamp=2024-08-04T12:00:01Z source_ip=192.168.1.10
destination_ip=192.168.1.20 username=admin event=LoginSuccess
description="User login successful"
```

Breakdown of the message

Key-Value Pairs: Data is presented as individual pairs separated by spaces.

- Timestamp: `timestamp=2024-08-04T12:00:01Z`
- Source IP: `source_ip=192.168.1.10`
- Destination IP: `destination_ip=192.168.1.20`
- Username: `username=admin`
- Event: `event=LoginSuccess`
- Description: `description="User login successful"`

LEEF (Log Event Extended Format)

LEEF is a log format developed for IBM QRadar. It is similar to CEF but with slight variations in structure and field definitions. There are two versions of LEEF. In LEEF 2.0, the delimiters between key-value pairs are configurable. For more information about LEEF, see LEEF event components.

Example of a LEEF-formatted log message

```
LEEF:2.0|SecurityVendor|Product|1.0|100|devTime=2024-08-04T12:00:01Z
src=192.168.1.10 dst=192.168.1.20 usrName=admin event=LoginSuccess
msg=User login successful
```

Breakdown of the message

Version Information: `LEEF:2.0` (indicates the version of LEEF).

Device Information:

- Vendor: `SecurityVendor`
- Product: `Product`

- Version: `1.0`

Event Information:

- Event ID: `100`

Key-Value Pairs: Presented in an extension format, similar to CEF.

- Device Time: `devTime=2024-08-04T12:00:01Z`
- Source IP: `src=192.168.1.10`
- Destination IP: `dst=192.168.1.20`
- User Name: `usrName=admin`
- Event: `event=LoginSuccess`
- Message: `msg=User login successful`

Syslog RFC 3164

RFC 3164 defines a traditional syslog format that includes mandatory header fields for a priority value, timestamp, and hostname followed by the rest of the message. It is less structured and often used for general logging purposes. For more information about syslog RFC 3164, see The BSD syslog Protocol.

Example of a log message with syslog RFC 3164 formatting

```
<29>Jul 4 13:54:45 radius-wifi radiusd[871]: (2571349) Login OK:
[tabletf] (from client wlc0-physical port 13 cli 90-06-28-ad-e8-2f)
```

Breakdown of the message

- **Priority Value**: `<29>` (encodes facility and severity).
- **Timestamp**: `Jul 4 13:54:45` (no year, no timezone).
- **Hostname**: `radius-wifi`
- **(Optional) Application/Process Name**: `radiusd`
- **(Optional) Process ID**: `871`
- **Message Content**: `(2571349) Login OK: [tabletf] (from client wlc0-physical port 13 cli 90-06-28-ad-e8-2f)`

Syslog RFC 5424

RFC 5424 is a more modern and structured syslog format, allowing for additional fields and structured data. It's used for more detailed and flexible logging. For more information about syslog RFC 5424, see The Syslog Protocol.

Example of a log message with syslog RFC 5424 formatting

```
<165>1 2024-08-04T12:00:01Z server1 appname 12345 ID47
[exampleSDID@32473 event="LoginSuccess" user="admin"
src_ip="192.168.1.10" dst_ip="192.168.1.20"] User login successful
```

Breakdown of the message

Priority Value: `<165>` (encodes facility and severity).

Version: 1 (indicates syslog version).

Timestamp: `2024-08-04T12:00:01Z`

Hostname: `server1` (can be empty)

Application/Process Name: `appname` (can be empty)

Process ID: `12345` (can be empty)

Message ID: `ID47` (can be empty)

Structured Data: `[exampleSDID@32473 event="LoginSuccess" user="admin" src_ip="192.168.1.10" dst_ip="192.168.1.20"]` (can be empty)

This is a structured data element, which provides additional context or metadata about the log message.

- SD-ID: `exampleSDID@32473` (This identifies the structured data element.)
- Event Field: `event="LoginSuccess"`
- User Field: `user="admin"`
- Source IP Field: `src_ip="192.168.1.10"`
- Destination IP Field: `dst_ip="192.168.1.20"`

Message Content: `User login successful`

Breakdown of the message

Priority Value: `<165>` (encodes facility and severity).

Version: 1 (indicates syslog version).

Timestamp: `2024-08-04T12:00:01Z`

Hostname: `server1`

Application/Process Name: `appname`

Process ID: `12345`

Message ID: `ID47`

Structured Data: `[exampleSDID@32473 event="LoginSuccess" user="admin" src_ip="192.168.1.10" dst_ip="192.168.1.20"]`

Message Content: `User login successful`

> If a field is empty, there should be a dash (-) rather than a completely blank field. This helps maintain the structure of the message and ensures that parsers can correctly interpret the fields. For example, if the `hostname` field is empty, the syslog message should look like this: `<165>1 2024-08-04T12:00:01Z - appname 12345: User login successful`

Summary of Differences

- **CEF, LEEF, and syslog (RFC 3164 & RFC 5424)** formats are primarily used in security logging and SIEMs.
- **CSV, TSV, pipe-separated values and JSON** are general-purpose formats, with JSON providing more structure and flexibility.
- **Key-Value Pairs** are simple and versatile but lack a standardized format.

Log Format Combinations

Log format types can be combined with each other and with other types of techniques for collecting, organizing, parsing, and pattern-matching data such as sFlow, XML, and regex (regular expression). Logstash is not a data log format either but a data processing pipeline that ingests, transforms, and sends data from various sources to destinations like Elasticsearch or other databases.

Example combining syslog RFC 3164 + local header + CEF header + message with JSON

```
<34>Aug 4 12:00:01 server1 localHeaderInfo:
CEF:0|SecurityVendor|Product|1.0|100|LoginSuccess|5|src=192.168.1.10
dst=192.168.1.20 suser=admin msg=User login successful |
{"timestamp": "2024-08-04T12:00:01Z", "source_ip": "192.168.1.10",
"destination_ip": "192.168.1.20", "username": "admin", "event":
"LoginSuccess", "description": "User login successful"}
```

Breakdown of the message

Syslog RFC 3164 Header:

- Priority Value: `<34>` (encodes facility and severity)
- Timestamp: `Aug 4 12:00:01`
- Hostname: `server1`

Local Header:

- Local Header Information: `localHeaderInfo` (this could be any additional information specific to the local system or application).

CEF Header:

- CEF: `0|SecurityVendor|Product|1.0|100|LoginSuccess|5|src=192.168.1.10 dst=192.168.1.20 suser=admin msg=User login successful`
- Details:
    - CEF Version: `CEF:0`
    - Vendor: `SecurityVendor`
    - Product: `Product`
    - Version: `1.0`
    - Signature ID: `100`
    - Event Name: `LoginSuccess`
    - Severity: `5`

- Key-Value Pairs:
    - Source IP: `src=192.168.1.10`
    - Destination IP: `dst=192.168.1.20`
    - Username: `suser=admin`
    - Message: `msg=User login successful`

Message with JSON: `{"timestamp": "2024-08-04T12:00:01Z", "source_ip": "192.168.1.10", "destination_ip": "192.168.1.20", "username": "admin", "event": "LoginSuccess", "description": "User login successful"}`

© 2026 Stellar Cyber . All rights reserved.